SPRINT 4

Modelat SQL

Erik Subirats Cruz

NIVELL 1 ⭐

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

En primer lloc, he creat la base de dades *transactions_sprint4*, que contindrà totes les taules necessàries per realitzar els exercicis:

```
2 •  CREATE DATABASE IF NOT EXISTS transactions_sprint4;
3 •   USE transactions_sprint4;
```

Output

Action Output

#	Time	Action	Message
1	18:10:07	CREATE DATABASE IF NOT EXISTS transactions_sprint4	1 row(s) affected

Creo la taula *companies*:

```
6 •  CREATE TABLE IF NOT EXISTS companies (
7        id VARCHAR(15) PRIMARY KEY,
8        company_name VARCHAR(255),
9        phone VARCHAR(15),
10       email VARCHAR(100),
11       country VARCHAR(100),
12       website VARCHAR(255)
13   );
```

Output

Action Output

#	Time	Action	Message
1	18:14:02	CREATE TABLE IF NOT EXISTS companies (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(15), ...	0 row(s) affected

Durant el procés d'importació de dades mitjançant la instrucció LOAD DATA LOCAL INFILE, es va detectar que MySQL tenia restringida la càrrega de fitxers a determinades carpetes del sistema. Per solucionar aquesta limitació, vaig haver de modificar el fitxer de configuració del servidor MySQL (my.ini), situat a C:\ProgramData\MySQL\MySQL Server 8.0\.

En aquest fitxer vaig ajustar els paràmetres relacionats amb la càrrega de fitxers per permetre importar arxius CSV des de qualsevol carpeta del sistema. Després de realitzar aquesta modificació, vaig reiniciar el servidor MySQL perquè els canvis tinguessin efecte. Finalment, vaig comprovar que la configuració s'havia aplicat correctament utilitzant les instruccions següents:

```
15 •  SHOW GLOBAL VARIABLES LIKE 'local_infile';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Variable_name	Value
local_infile	ON

Output

Action Output

#	Time	Action	Message
1	19:42:40	SHOW GLOBAL VARIABLES LIKE 'local_infile'	1 row(s) returned

```
16 •  SHOW VARIABLES LIKE 'secure_file_prive';
17
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Variable_name	Value

Result 2

Output

Action Output

#	Time	Action	Message
1	19:42:40	SHOW GLOBAL VARIABLES LIKE 'local_infile'	1 row(s) returned
2	19:43:27	SHOW VARIABLES LIKE 'secure_file_prive'	0 row(s) returned

Ara ja puc carregar les dades des de qualsevol carpeta:

```
19 •  LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/companies.csv'
20    INTO TABLE companies
21    FIELDS TERMINATED BY ','
22    ENCLOSED BY '"'
23    LINES TERMINATED BY '\n'
24    IGNORE 1 LINES
25    (id, company_name, phone, email, country, website);
```

Output

Action Output

#	Time	Action	Message
1	19:48:15	LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/companies.csv' INTO TABLE companies FIELDS TERMINATED BY ',' ENCLOS...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Creo la taula *credit_cards* i carrego les dades:

```
28 •  CREATE TABLE IF NOT EXISTS credit_cards (
29        id VARCHAR(15) PRIMARY KEY UNIQUE,
30        user_id INT,
31        iban VARCHAR(50),
32        pan VARCHAR(20),
33        pin VARCHAR(4),
34        cvv VARCHAR(3),
35        track1 VARCHAR(100),
36        track2 VARCHAR(100),
37        expiring_date VARCHAR(8)
38    );
```

Output

Action Output

#	Time	Action	Message
1	19:59:15	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) PRIMARY KEY UNIQUE, user_id INT, iban VARCHAR(50), pan VARCHAR(...	0 row(s) affected

```sql
LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/credit_cards.csv'
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, user_id, iban, pan, pin, cw, track1, track2, expiring_date);
```

Output

Action Output

#	Time	Action	Message
1	20:01:08	LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/credit_cards.csv' INTO TABLE credit_cards FIELDS TERMINATED BY ',' ENCLO...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

Inicialment, creo dues taules separades, *european_users* i *american_users* i carrego les dades:

```sql
CREATE TABLE IF NOT EXISTS european_users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(150),
    email VARCHAR(150),
    birth_date VARCHAR(100),
    country VARCHAR(150),
    city VARCHAR(150),
    postal_code VARCHAR(100),
    address VARCHAR(255)
);
```

Output

Action Output

#	Time	Action	Message
1	11:52:24	CREATE TABLE IF NOT EXISTS european_users (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(150), em...	0 row(s) affected

```sql
#Insertar dades a taula 'european_users'
LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/european_users.csv'
INTO TABLE european_users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, name, surname, phone, email, birth_date, country, city, postal_code, address);
```

Output

Action Output

#	Time	Action	Message
1	12:06:57	LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/european_users.csv' INTO TABLE european_users FIELDS TERMINATED BY ',' ...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0

```sql
CREATE TABLE IF NOT EXISTS american_users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(150),
    email VARCHAR(150),
    birth_date VARCHAR(100),
    country VARCHAR(150),
    city VARCHAR(150),
    postal_code VARCHAR(100),
    address VARCHAR(255)
);
```

Output

Action Output

#	Time	Action	Message
1	11:55:55	CREATE TABLE IF NOT EXISTS american_users (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(150), emai...	0 row(s) affected

```
85  LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/american_users.csv'
86  INTO TABLE american_users
87  CHARACTER SET utf8mb4
88  FIELDS TERMINATED BY ','
89  ENCLOSED BY '"'
90  LINES TERMINATED BY '\n'
91  IGNORE 1 LINES
92  (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
```

Posteriorment creo una taula comuna anomenada *users*, amb la mateixa estructura. Les dades de les taules *european_users* i *american_users* són inserides en aquesta nova taula mitjançant dues instruccions INSERT INTO i SELECT.

```
97  CREATE TABLE IF NOT EXISTS users (
98      id INT PRIMARY KEY,
99      name VARCHAR(100),
100     surname VARCHAR(100),
101     phone VARCHAR(150),
102     email VARCHAR(150),
103     birth_date VARCHAR(100),
104     country VARCHAR(150),
105     city VARCHAR(150),
106     postal_code VARCHAR(100),
107     address VARCHAR(255)
108 );
```

Output

Action Output

#	Time	Action	Message
1	12:02:51	CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(150), email VARCHA...	0 row(s) affected

```
110 INSERT INTO users
111     SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address
112     FROM european_users;
113 INSERT INTO users
114     SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address
115     FROM american_users;
```

Output

Action Output

#	Time	Action	Message
1	12:08:34	INSERT INTO users SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address FROM european_users	3990 row(s) affected Records: 3990 Duplicates: 0 Warnings: 0
2	12:08:37	INSERT INTO users SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address FROM american_users	1010 row(s) affected Records: 1010 Duplicates: 0 Warnings: 0

Un cop unificades les dades, les taules originals es van eliminar per simplificar l'estructura de la base de dades.

```
118 DROP TABLE IF EXISTS american_users;
119 DROP TABLE IF EXISTS european_users;
```

Output

Action Output

#	Time	Action	Message
1	12:10:27	DROP TABLE IF EXISTS american_users	0 row(s) affected
2	12:10:32	DROP TABLE IF EXISTS european_users	0 row(s) affected

Creo la taula *products* i carrego les dades:

```sql
CREATE TABLE IF NOT EXISTS products (
    id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price VARCHAR(100),
    colour VARCHAR(150),
    weight FLOAT,
    warehouse_id CHAR(10)
);
```

Output

Action Output

#	Time	Action	Message
1	12:11:49	CREATE TABLE IF NOT EXISTS products (id INT PRIMARY KEY, product_name VARCHAR(100), price VARCHAR(100), colour VARCHAR(150), weight...	0 row(s) affected

```sql
LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, product_name, price, colour, weight, warehouse_id);
```

Output

Action Output

#	Time	Action	Message
1	16:26:04	LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/products.csv' INTO TABLE products FIELDS TERMINATED BY ',' ENCLOSED B...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Finalment, creo la taula *transactions* i carrego les dades:

```sql
CREATE TABLE IF NOT EXISTS transactions (
    id VARCHAR(255) PRIMARY KEY,
    card_id VARCHAR(15),
    business_id VARCHAR(15),
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN,
    product_ids VARCHAR(100),
    user_id INT,
    lat FLOAT,
    longitude FLOAT,
    FOREIGN KEY (business_id) REFERENCES companies(id),
    FOREIGN KEY (card_id) REFERENCES credit_cards(id),
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Output

Action Output

#	Time	Action	Message
1	20:09:32	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), ti...	0 row(s) affected

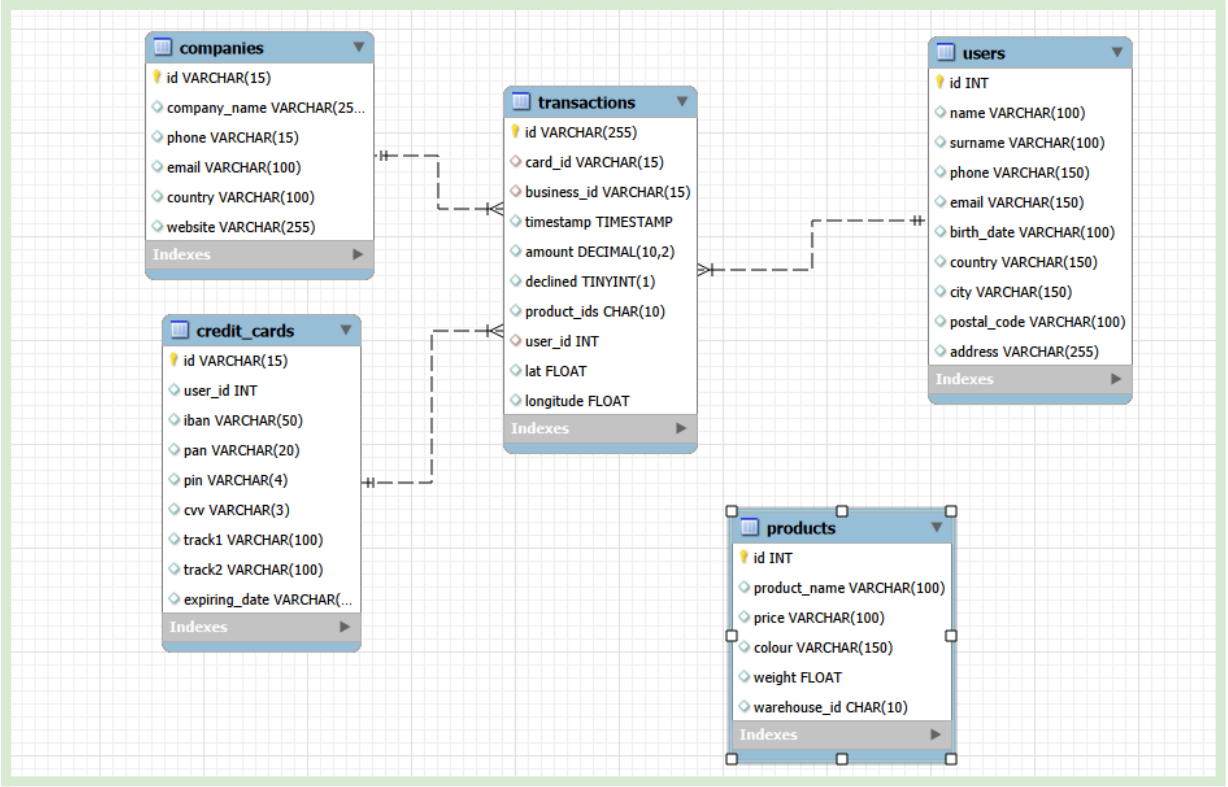
```sql
LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude);
```

Output

Action Output

#	Time	Action	Message
1	20:09:32	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), ti...	0 row(s) affected
2	20:10:09	LOAD DATA LOCAL INFILE 'C:/Users/eriks/Desktop/ADD/SPRINT 4/transactions.csv' INTO TABLE transactions FIELDS TERMINATED BY ',' LINES ...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

Aquesta taula inclou diverses claus foranes per relacionar-se amb les taules *companies*, *credit_cards* i *users*, garantint la integritat referencial de les dades. La relació amb la taula *products* la veurem al Nivell 3.



Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
164 •  SELECT
165      u.id,
166      u.name,
167      u.surname
168    FROM users u
169    WHERE u.id IN (
170      SELECT t.user_id
171      FROM transactions t
172      GROUP BY t.user_id
173      HAVING COUNT(t.id) > 80
174    );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	surname
185	Molly	Gilliam
289	Dxwqi	Hwcru
318	Bnyr	Astuw
454	Sfzzoh	Xqvfridxs
NULL	NULL	NULL

users 5

Output

Action Output

#	Time	Action	Message
1	16:41:48	SELECT u.id, u.name, u.surname FROM users u WHERE u.id IN (SELECT t.user_id FROM transactions t GROUP BY t.user_id HAVING COU...	4 row(s) returned

Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```sql
SELECT
  cc.iban AS IBAN,
  AVG(t.amount) AS avg_amount
FROM credit_cards cc
JOIN transactions t ON cc.id = t.card_id
JOIN companies c ON t.business_id = c.id
WHERE c.company_name = 'Donec Ltd'
  AND t.declined = 0
GROUP BY cc.id, cc.iban
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

IBAN	avg_amount
XX9114064011255863075868 05	356.246667
SK9446370242474562577506	142.960000
XX7767529178459529755556 40	257.370000
XX4138273622897193049089 90	139.590000
XX3477872460707696107803 08	240.410000
XX6887684365430908948546 02	188.580000
MK28368851538688349	439.390000
PL76249283566852676343404576	541.560000
XX8040083311349183418039 13	189.210000
LB64655537777363327873049938	155.500000
XX3545127837558213810084 45	255.830000

Result 10

Output

Action Output

#	Time	Action	Message
1	16:49:59	SELECT cc.iban AS IBAN, AVG(t.amount) AS avg_amount FROM credit_cards cc JOIN transactions t ON cc.id = t.card_id JOIN companies c ON t.busi...	370 row(s) returned

NIVELL 2 ★★

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

Creo la taula *card_activity*:

```sql
CREATE TABLE IF NOT EXISTS card_activity (
  card_id VARCHAR(15) PRIMARY KEY,
  activity VARCHAR(10)
);
```

Output

Action Output

#	Time	Action	Message
1	19:35:07	CREATE TABLE IF NOT EXISTS card_activity (card_id VARCHAR(15) PRIMARY KEY, activity VARCHAR(10))	0 row(s) affected

Per determinar si una targeta és activa o inactiva, analitzo les tres transaccions més recents de cada targeta utilitzant la funció ROW_NUMBER() amb una finestra ordenada per data (*timestamp*). Si la suma de les tres transaccions més recents d'una targeta han estat rebutjades (*declined* = 1), el resultat és SUM(declined) = 3 i la targeta es considera INACTIVE. En cas contrari, es considera ACTIVE.

```sql
INSERT INTO card_activity (card_id, activity)
SELECT
  card_id,
  CASE
    WHEN SUM(declined) = 3 THEN 'INACTIVE'
    ELSE 'ACTIVE'
  END AS activity
FROM (
  SELECT
    card_id,
    declined,
    ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) AS rn
  FROM transactions
) t
WHERE rn IN (1, 2, 3)
GROUP BY card_id;
```

Output

Action Output

#	Time	Action	Message
1	19:39:02	INSERT INTO card_activity (card_id, activity) SELECT card_id, CASE WHEN SUM(declined) = 3 THEN 'INACTIVE' ELSE 'ACTIVE' E...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

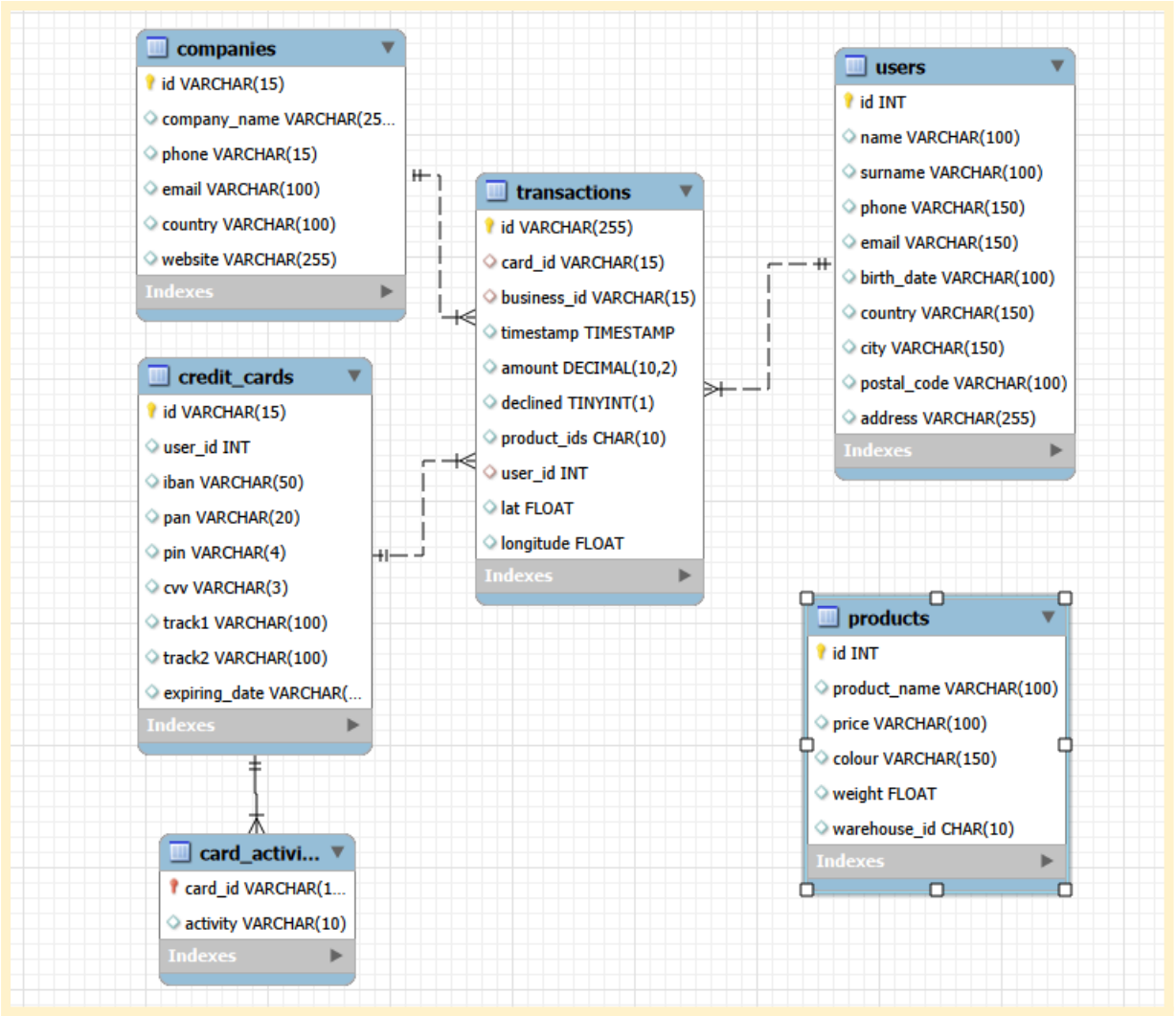
Posteriorment es va establir una FK amb la taula *credit_cards*.

```sql
ALTER TABLE card_activity
ADD FOREIGN KEY (card_id) REFERENCES credit_cards(id);
```

Output

Action Output

#	Time	Action	Message
1	19:40:06	ALTER TABLE card_activity ADD FOREIGN KEY (card_id) REFERENCES credit_cards(id)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0



Exercici 1

Quantes targetes estan actives?

```
SELECT COUNT(*) AS active_cards
FROM card_activity
WHERE activity = 'ACTIVE';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

active_cards
4995

Result 6

Output

Action Output

#	Time	Action	Message
1	11:45:55	SELECT COUNT(*) AS active_cards FROM card_activity WHERE activity = 'ACTIVE'	1 row(s) returned

NIVELL 3 ⭐⭐⭐

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Creo la taula *transaction_products* destinada a representar la relació entre transaccions i productes. Aquesta taula conté l'identificador de la transacció i l'identificador del producte, i estableix FK amb les taules *transactions* i *products*.

```
CREATE TABLE transaction_products (
    transaction_id VARCHAR(255),
    product_id INT,
    FOREIGN KEY (transaction_id) REFERENCES transactions(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

Output

Action Output

#	Time	Action	Message
1	20:01:25	CREATE TABLE transaction_products (transaction_id VARCHAR(255), product_id INT, FOREIGN KEY (transaction_id) REFERENCES transactio...	0 row(s) affected

A la taula *transactions*, la columna *product_ids* conté diversos identificadors de producte dins d'una mateixa cel·la separats per comes (per exemple, '87, 9, 34'). Aquest format no és adequat per a una base de dades relacional. Abans de separar els productes de la columna *product_ids*, he realitzat una consulta per identificar la cadena amb més productes. Per fer-ho he utilitzat la funció CHAR_LENGTH, que permet calcular la longitud del text. Agafant el resultat més llarg, he pogut saber quin era el nombre màxim de productes per transacció (5):

```
SELECT
    product_ids,
    CHAR_LENGTH(product_ids) AS longitud
FROM transactions
ORDER BY longitud DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_ids	longitud
94, 43, 52, 79, 100	19

Result 24

Output

Action Output

#	Time	Action	Message
1	20:12:09	SELECT product_ids, CHAR_LENGTH(product_ids) AS longitud FROM transactions ORDER BY longitud DESC LIMIT 1	1 row(s) returned

Per inserir les dades a la taula *transaction_products*, he creat una subconsulta amb UNION ALL per extreure cada producte per separat i he utilitzat diverses funcions:

- SUBSTRING_INDEX(product_ids, ',', n) agafa els primers n valors separats per comes.
- SUBSTRING_INDEX(..., ',', -1) s'utilitza per agafar només l'últim valor de l'extracció anterior, obtenint així exactament el producte n.
- TRIM(...) elimina espais en blanc que poguessin quedar al voltant dels números de producte.

El UNION ALL permet combinar totes les extraccions (de l'1r fins al 5è producte) en un sol conjunt, i finalment SELECT DISTINCT garanteix que no hi hagi duplicats abans d'inserir-les a la taula.

```sql
INSERT INTO transaction_products
SELECT DISTINCT transaction_id, product_id
FROM (
    SELECT id AS transaction_id, TRIM(SUBSTRING_INDEX(product_ids, ',', 1)) AS product_id
    FROM transactions

    UNION ALL

    SELECT id AS transaction_id, TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 2), ',', -1)) AS product_id
    FROM transactions

    UNION ALL

    SELECT id AS transaction_id, TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 3), ',', -1)) AS product_id
    FROM transactions

    UNION ALL

    SELECT id AS transaction_id, TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 4), ',', -1)) AS product_id
    FROM transactions

    UNION ALL

    SELECT id AS transaction_id, TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 5), ',', -1)) AS product_id
    FROM transactions
) AS t;
```

Output

Action Output

#	Time	Action	Message
1	19:11:05	INSERT INTO transaction_products SELECT DISTINCT transaction_id, product_id FROM (SELECT id AS transaction_id, TRIM(SUBSTRING_INDEX...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0

Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

```sql
SELECT
    p.product_name,
    COUNT(*) AS total_sales
FROM transaction_products tp
JOIN products p ON tp.product_id = p.id
JOIN transactions t ON tp.transaction_id = t.id
WHERE declined = 0
GROUP BY tp.product_id, p.product_name
ORDER BY total_sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	total_sales
riverlands the duel	2642
Tully maester Tarly	2627
duel Direwolf	2603
the duel warden	2602
duel warden	2593
sith Jade	2591
mustafar jinn	2590
rock Renly in	2589
Stark Karstark	2587
riverlands north	2586
Stannis warden so...	2582
Dorne bastard	2579
of north	2574
warden south duel	2573

Result 2

Output

Action Output

#	Time	Action	Message
1	11:42:45	SELECT p.product_name, COUNT(*) AS total_sales FROM transaction_products tp JOIN products p ON tp.product_id = p.id JOIN transactions t ON tp.t...	100 row(s) returned